# LLM-Based Threat Modeling System

## Complete Python Code Reference

Detailed Explanation of Every Function, Library & Library Method

Group 10 | University Project | 2024

Based on: Elsharef, Zeng & Gu — AISCC Workshop @ NDSS 2024

# 1. Project Overview

This document provides a comprehensive, line-by-line explanation of all Python functions, external libraries, and individual library methods used across the four implementation files of the LLM-Based Threat Modeling System.

The system is organized into four Python scripts implementing two tasks, each with two approaches:

| File | Purpose |
|---|---|
| `task1_base_llm.py` | Task 1 — MQ1 using Base LLM without RAG |
| `task1_rag_llm.py` | Task 1 — MQ1 using RAG-Enhanced LLM with Pinecone/FAISS |
| `task2_base_llm.py` | Task 2 — MQ2 using Base LLM with no NVD lookup |
| `task2_rag_nvd.py` | Task 2 — MQ2 using RAG + NVD full pipeline (KeyBERT → NVD → JSONL → Pinecone → Llama 2) |
| `demo_runner.py` | Unified runner that executes all four approaches sequentially |

# 2. Libraries — Detailed Explanation

Each library, its purpose, installation command, and every method used in the project is explained below.

## 2.1 transformers (HuggingFace Transformers)

| Install | pip install transformers |
|---|---|

The HuggingFace Transformers library is the backbone of modern NLP. It provides thousands of pre-trained models (BERT, GPT, Llama, T5, etc.) with a unified Python API. It handles model downloading, tokenization, inference, and fine-tuning. In this project it loads the Llama 2 Chat model for text generation.

### Library Functions / Classes Used

| Method / Class | Detailed Explanation |
|---|---|
| `AutoTokenizer .from_pretrained(model_id)` | Automatically detects the correct tokenizer class for a given model identifier (e.g., 'meta-llama/Llama-2-7b-chat-hf'). Downloads the tokenizer configuration, vocabulary files, and special token mappings from HuggingFace Hub. Returns a tokenizer object capable of converting raw text into token IDs that the model understands. The 'Auto' prefix means it reads the model config to decide which tokenizer subclass to instantiate. |
| `AutoModelForCausalLM .from_pretrained(model_id, torch_dtype=..., device_map='auto', load_in_8bit=...)` | Loads a causal language model (a decoder-only model that predicts the next token). 'Causal' means each token only attends to previous tokens — perfect for text generation. torch_dtype=torch.float16 uses half-precision to halve GPU memory. device_map='auto' automatically distributes model layers across available GPUs/CPUs. load_in_8bit=True uses bitsandbytes to quantize weights to 8-bit integers, reducing memory by ~4x with minimal quality loss. |
| `pipeline('text-generation', model=..., tokenizer=..., max_new_tokens=..., temperature=..., do_sample=..., repetition_penalty=...)` | Creates a high-level inference pipeline that wraps the model + tokenizer into one callable object. 'text-generation' means the pipeline generates new text given a prompt. max_new_tokens caps how many tokens the model generates (prevents runaway output). temperature controls randomness: 0.0 = deterministic, 1.0 = very random; we use 0.1 for factual security answers. do_sample=True enables stochastic sampling (required when temperature < 1). repetition_penalty > 1.0 penalises repeating the same tokens, reducing verbose loops. |

#### Llama-2-Chat Prompt Template

Llama 2 Chat models expect a specific prompt format. Using the wrong format degrades output quality significantly:

```
<s>[INST] <<SYS>>
You are a security expert. Answer concisely.
<</SYS>>
```

```
Context: {retrieved_document_chunks}

Question: {user_question} [/INST]
```

The [INST] tags delimit the instruction, <<SYS>> tags wrap the system-level persona, and [/INST] marks where the model should begin its answer.

## 2.2 torch (PyTorch)

| Install | pip install torch (or torch+cuda for GPU support) |
|---|---|

PyTorch is the deep learning framework underlying HuggingFace Transformers. It provides tensors (multi-dimensional arrays on CPU/GPU), automatic differentiation, and neural network primitives. In this project PyTorch is used for GPU detection and dtype configuration.

| Method / Attribute | Detailed Explanation |
|---|---|
| `torch.cuda.is_available()` | Returns True if CUDA (NVIDIA GPU) is available and properly configured. Used to conditionally enable GPU-specific optimizations like float16 dtype and 8-bit quantization. On a CPU-only machine this returns False and the code falls back to float32. |
| `torch.float16` | A PyTorch data type constant representing 16-bit floating point numbers (half precision). Using float16 for model weights halves GPU memory consumption compared to float32 (default). Slightly less numerically precise but acceptable for inference. Only supported on CUDA-capable devices. |
| `torch.float32` | Standard 32-bit floating point (single precision). The safe default for CPU inference where float16 is not supported. Uses twice the memory of float16 but is universally compatible across all hardware. |

## 2.3 langchain (LangChain Framework)

| Install | pip install langchain langchain-community |
|---|---|

LangChain is a framework for building applications powered by LLMs. It provides standard interfaces for document loading, text splitting, embedding models, vector stores, and chains (sequences of LLM calls). It abstracts away the boilerplate of connecting different AI components together.

### 2.3.1 Document Loaders

| Class / Method | Detailed Explanation |
|---|---|
| `PyPDFDirectoryLoader (pdf_dir)` | Scans an entire directory for .pdf files and loads them all. Internally uses PyPDF2/pypdf to extract text page by page. Returns a list of Document objects where each Document has a page_content string and a metadata dict containing 'source' (filename) and 'page' (page number). The directory scan means adding new PDFs requires no code changes — just drop files into the folder. |

| Class / Method | Detailed Explanation |
|---|---|
| `loader.load()` | Executes the loading process and returns all documents as a Python list. Each element is a langchain Document object. A 60-page PDF becomes 60 Document objects, one per page. This lazy loading pattern separates configuration from execution. |

### 2.3.2 Text Splitters

| Class / Method | Detailed Explanation |
|---|---|
| `RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=20, separators=[...])` | Splits documents into smaller chunks for embedding. 'Recursive' means it tries each separator in order: first splits on double newlines (paragraph boundaries), then single newlines, then spaces, then individual characters — only going smaller if the chunk is still too large. This preserves natural language boundaries. chunk_size=500 characters balances context richness vs. embedding quality. chunk_overlap=20 characters means adjacent chunks share 20 characters, preventing context loss at boundaries. |
| `splitter.split_documents(docs)` | Applies the splitting strategy to a list of Document objects. Returns a new list of Document objects where each represents one chunk. Importantly, metadata (source filename, page number) is copied to all child chunks, so answers can be traced back to their source document and page. |

### 2.3.3 Embeddings

| Class / Method | Detailed Explanation |
|---|---|
| `HuggingFaceEmbeddings(model_name='sentence-transformers/all-MiniLM-L6-v2')` | Wraps a sentence-transformers model from HuggingFace into LangChain's embedding interface. all-MiniLM-L6-v2 is a 22M parameter model that produces 384-dimensional dense vectors. It's optimized for semantic similarity tasks: sentences with similar meaning produce vectors that are close in cosine distance. Chosen for its balance of speed (fast inference), quality (strong similarity scores), and size (fits on CPU). |
| `embeddings.embed_documents(texts)` | Converts a list of text strings into a list of 384-dimensional float vectors. Called internally by vector stores during indexing. The resulting vectors capture semantic meaning — 'TLS handshake' and 'SSL connection setup' would have similar vectors even though they share no words. |
| `embeddings.embed_query(text)` | Converts a single query string to a vector. Called at retrieval time to find similar document chunks. The same model must be used for both indexing (embed_documents) and querying (embed_query) to ensure vectors are in the same semantic space. |

### 2.3.4 Vector Stores

| Class / Method | Detailed Explanation |
|---|---|
| `FAISS .from_documents(docs, embeddings)` | Creates an in-memory FAISS index from a list of Document objects. FAISS (Facebook AI Similarity Search) uses optimized algorithms for |

| Class / Method | Detailed Explanation |
|---|---|
| | fast nearest-neighbour search in high-dimensional spaces. Internally embeds all documents, creates a flat L2 or cosine index, and stores text + metadata. No API key required — runs entirely locally. Best for development and small-to-medium datasets. |
| `Pinecone .from_documents( docs, embeddings, index_name=...)` | Pushes documents to a cloud-hosted Pinecone index. Pinecone is a managed vector database that scales to billions of vectors with millisecond query latency. Requires PINECONE_API_KEY. Supports metadata filtering, namespaces, and pod-based or serverless deployment. The cloud hosting means the index persists between code runs and can be shared across team members. |
| `vector_store .as_retriever( search_kwargs={'k':5})` | Converts a vector store into a LangChain Retriever object. The retriever has a single get_relevant_documents(query) method that embeds the query and returns the top-k most similar documents. k=5 balances between giving the LLM enough context and avoiding noise from less-relevant chunks. The retriever abstraction allows swapping FAISS for Pinecone transparently. |
| `retriever .get_relevant_documents (question)` | The primary retrieval call. Embeds the question text, performs cosine similarity search against all indexed chunks, and returns the top-k Document objects. Each returned document includes page_content (the chunk text) and metadata (source, page, CVE ID, severity, etc.). These documents become the 'context' injected into the LLM prompt. |

## 2.4 keybert (Keyword Extraction)

| Install | pip install keybert |
|---|---|

KeyBERT uses BERT-based embeddings and cosine similarity to extract keywords that are most representative of a document. Unlike TF-IDF (which counts word frequency), KeyBERT finds words whose semantic meaning matches the overall document meaning. This produces more contextually relevant keywords for NVD API queries.

| Method / Parameter | Detailed Explanation |
|---|---|
| `KeyBERT( model='sentence-transformers/ all-MiniLM-L6-v2')` | Instantiates the KeyBERT extractor using a specified sentence-transformer model. The model converts both the full document and individual candidate keywords into embedding vectors. If model is not specified, KeyBERT defaults to 'all-MiniLM-L6-v2'. Using the same embedding model as the vector store ensures consistency in semantic space. |
| `kw_model .extract_keywords( text, keyphrase_ngram_range=(1,2), stop_words=list(all_stops), top_n=15, use_mmr=True, diversity=0.5)` | The core extraction function. text is the full document string. keyphrase_ngram_range=(1,2) extracts both single words and two-word phrases (bigrams), providing more context than single words alone. stop_words removes the provided list from candidates before scoring. top_n=15 returns the 15 highest-scoring keywords. Returns a list of (keyword, score) tuples sorted by relevance score. |
| `use_mmr=True` | Enables Maximal Marginal Relevance, a diversity algorithm that balances relevance (similarity to document) with novelty (dissimilarity to already-selected keywords). Without MMR, extracted keywords |

| Method / Parameter | Detailed Explanation |
|---|---|
| | tend to cluster around one topic. With MMR, keywords spread across the document's themes. Controlled by the diversity parameter. |
| `diversity=0.5` | Controls the MMR diversity-relevance trade-off. 0.0 = purely relevance-focused (keywords may be redundant). 1.0 = purely diversity-focused (keywords may be off-topic). 0.5 gives equal weight to both, producing a varied but still relevant keyword set. |

## 2.5 nltk (Natural Language Toolkit)

| Install | pip install nltk |
|---|---|

NLTK is one of the oldest and most comprehensive Python NLP libraries. It provides tools for tokenization, stemming, lemmatization, part-of-speech tagging, parsing, and corpora access. In this project it provides two specific features: stopword lists and the Porter Stemmer.

| Method / Class | Detailed Explanation |
|---|---|
| `nltk.download('stopwords', quiet=True)` | Downloads the NLTK stopwords corpus to the local machine (~100KB). quiet=True suppresses the download progress message. Must be called before accessing stopwords. Safe to call repeatedly — NLTK skips the download if the corpus is already present. |
| `nltk.download('punkt', quiet=True)` | Downloads the Punkt tokenizer models, which are required for sentence and word tokenization. Although not used directly in our main pipeline, it is a common dependency that prevents import errors from downstream NLTK operations. |
| `stopwords.words('english')` | Returns a list of 179 common English stopwords ('the', 'a', 'is', 'in', 'and', etc.). These are words that appear so frequently in any text that they carry no discriminative information. Removing them before keyword extraction prevents generic words from appearing as 'keywords'. Accessed from nltk.corpus.stopwords after downloading the corpus. |
| `PorterStemmer()` | Implements the Porter stemming algorithm, which reduces words to their root form using a series of rule-based suffix stripping steps. Examples: 'encryption' → 'encrypt', 'encrypted' → 'encrypt', 'encrypting' → 'encrypt'. This ensures that 'TLS encryption' and 'TLS encrypted' produce the same NVD API keyword 'encrypt', preventing duplicate search results. |
| `stemmer.stem(word)` | Applies the Porter stemming rules to a single word string. Returns the stemmed root. Not linguistically perfect (produces 'comput' from 'computer'), but consistent and fast. Used here only for deduplication of NVD keyword searches, not for display purposes. |

## 2.6 sentence-transformers

| Install | pip install sentence-transformers |
|---|---|

The sentence-transformers library provides pre-trained transformer models specifically fine-tuned for producing high-quality sentence and paragraph embeddings. While transformers produces token-level representations, sentence-transformers produces a single fixed-size vector for an entire sentence or passage via pooling strategies.

| Method / Model | Detailed Explanation |
|---|---|
| `all-MiniLM-L6-v2` | A 6-layer MiniLM model fine-tuned on over 1 billion sentence pairs using contrastive learning (similar sentences are pulled together, dissimilar ones are pushed apart). Produces 384-dimensional embeddings. 'L6' means 6 transformer layers — much smaller than full BERT (12 layers), giving 5x faster inference with ~90% of the embedding quality. Used by both KeyBERT and HuggingFaceEmbeddings in this project. |
| `SentenceTransformer (model_name)` | Loads a sentence-transformer model from HuggingFace Hub or local cache. Encapsulates the model, tokenizer, and pooling strategy. The default pooling is mean pooling over all token embeddings, then L2 normalization, producing unit-length vectors suitable for cosine similarity. |
| `model.encode(sentences)` | Encodes a list of sentences into a NumPy array of shape (n_sentences, embedding_dim). Handles batching internally for efficiency. Called by HuggingFaceEmbeddings and KeyBERT under the hood. |

## 2.7 pinecone-client (Pinecone Vector Database)

| Install | pip install pinecone-client |
|---|---|

Pinecone is a managed, cloud-native vector database built specifically for similarity search at scale. Unlike FAISS (in-memory, single machine), Pinecone persists data in the cloud, supports billions of vectors, and provides millisecond-latency search across distributed infrastructure. It handles sharding, replication, and index maintenance automatically.

| Method / Function | Detailed Explanation |
|---|---|
| `pinecone.init( api_key=PINECONE_API, environment=PINECONE_ENV)` | Initializes the Pinecone client with authentication credentials. PINECONE_API is the secret API key from the Pinecone console. environment specifies the cloud region (e.g., 'us-east-1'). Must be called before any other Pinecone operations. Credentials are validated against the Pinecone service on first use. |
| `pinecone.list_indexes()` | Returns a list of existing index names in the account. Used to check whether the target index already exists before attempting to create it, preventing 'AlreadyExistsError' exceptions. |
| `pinecone.create_index( name, dimension=384, metric='cosine')` | Creates a new vector index. dimension=384 must match the embedding model output size (all-MiniLM-L6-v2 produces 384-dim vectors). metric='cosine' sets the similarity metric — cosine similarity measures the angle between vectors, making it scale-invariant (length doesn't affect similarity). Other options: 'euclidean' (L2 distance), 'dotproduct' (inner product). |

| Method / Function | Detailed Explanation |
|---|---|
| `Pinecone .from_documents( docs, embeddings, index_name=INDEX_NAME)` | LangChain's wrapper that embeds all documents and upserts them into the named Pinecone index. Each document becomes one vector with its text stored in metadata. The 'upsert' operation inserts new vectors and updates existing ones with matching IDs, making it idempotent. |

## 2.8 faiss-cpu (Facebook AI Similarity Search)

**Install** pip install faiss-cpu (or faiss-gpu for CUDA)

FAISS is a library developed by Meta AI Research for efficient similarity search and clustering of dense vectors. It implements multiple index types optimized for different trade-offs between memory, speed, and accuracy. faiss-cpu runs on CPU without any cloud dependency — ideal for development and smaller datasets.

| Operation | Detailed Explanation |
|---|---|
| `FAISS .from_documents( docs, embeddings)` | LangChain wrapper that embeds all documents and builds a flat FAISS index (IndexFlatL2 by default). 'Flat' means every query vector is compared against every stored vector — exact search with O(n) time complexity. For this project's scale (1,000–36,000 CVEs), flat search is fast enough (< 100ms). The index lives in RAM. |
| `FAISS .from_texts( texts, embeddings, metadatas=...)` | Alternative constructor that accepts raw text strings with optional metadata dicts instead of Document objects. Used when building the CVE knowledge base from JSONL entries directly. |
| `vector_store .similarity_search( query, k=5)` | Performs exact nearest-neighbour search. Embeds the query, computes L2 distance to all indexed vectors, returns k closest Documents. L2 distance is mathematically equivalent to cosine similarity for unit-norm vectors (which sentence-transformers produces after normalization). |
| `vector_store .save_local(path)` | Serializes the FAISS index and document store to disk. Allows reuse without re-embedding documents on the next run. Saves two files: index.faiss (binary index) and index.pkl (Python pickle of texts + metadata). |
| `FAISS .load_local( path, embeddings)` | Reconstructs a FAISS index from disk. The embedding model must be the same as used during save_local — otherwise the vector spaces won't match. |

## 2.9 requests (HTTP Library)

**Install** pip install requests

The requests library is the de-facto standard Python HTTP client. It provides a simple API for making GET, POST, PUT, DELETE requests with automatic JSON handling, session management, SSL verification, and timeouts. Used here to query the NVD (National Vulnerability Database) REST API v2.

| Method / Parameter | Detailed Explanation |
|---|---|
| `requests.get( url, params=params, headers=headers, timeout=10)` | Sends an HTTP GET request to the specified URL. params is a dict of query parameters that are URL-encoded and appended to the URL automatically. headers is a dict of HTTP headers (used to send the NVD API key). timeout=10 means the request raises requests.exceptions.Timeout if no response is received within 10 seconds, preventing the program from hanging indefinitely. |
| `response .raise_for_status()` | Inspects the HTTP status code and raises an HTTPError if the code indicates failure (4xx client errors or 5xx server errors). If the status code is 200 OK, does nothing. Provides one-line error handling for HTTP failures without needing to check response.status_code manually. |
| `response.json()` | Parses the response body as JSON and returns a Python dict/list. The NVD API returns a JSON object containing a 'vulnerabilities' array. This method internally calls json.loads() on response.text and handles encoding detection. |
| `requests.exceptions .RequestException` | Base class for all requests exceptions. Catching this covers: ConnectionError (network unavailable), Timeout (no response in time), HTTPError (4xx/5xx status), and TooManyRedirects. Used in the try/except block around NVD API calls to handle network failures gracefully. |

### NVD API Rate Limiting

The NVD API imposes rate limits that must be respected to avoid IP blocking:

```
# Without API key: 5 requests per 30 seconds
time.sleep(6.5)   # 30s / 5 = 6s minimum; 6.5s adds safety margin

# With API key: 50 requests per 30 seconds
time.sleep(0.6)   # 30s / 50 = 0.6s minimum
```

## 2.10 json (JSON Standard Library)

| Install | Built-in Python standard library — no installation required |
|---|---|

Python's built-in JSON module serializes Python objects to JSON strings and deserializes JSON strings back to Python objects. Used here for the JSONL (JSON Lines) vulnerability knowledge base format — one JSON object per line.

| Function | Detailed Explanation |
|---|---|
| `json.dumps(obj)` | Serializes a Python dict/list to a JSON-formatted string. Ensures all special characters are properly escaped. Used to write one CVE entry per line to the JSONL file. The resulting string contains no newlines within it (unless indent= is specified), making it safe for line-by-line file writing. |
| `json.loads(string)` | Parses a JSON-formatted string and returns a Python dict or list. Used when reading each line of the JSONL file back into Python objects. |

| Function | Detailed Explanation |
|---|---|
| | Raises json.JSONDecodeError if the string is malformed, which can be caught to handle corrupt entries without failing the entire file load. |

### Why JSONL Over Regular JSON?

JSONL (JSON Lines) stores one JSON object per line. This has three advantages over a single large JSON array:

- Stream processing: can read and process one CVE at a time without loading the entire file into memory
- Error isolation: a corrupt line only skips one CVE, not the entire dataset
- Space efficiency: no wrapping array brackets, commas between objects, or indentation needed

## 2.11 csv (CSV Standard Library)

| Install | Built-in Python standard library — no installation required |
|---|---|

Python's built-in csv module reads and writes comma-separated values files, handling quoting, escaping, and different delimiters automatically. Used here to save extracted keywords and CVE details to intermediate files between pipeline stages.

| Class / Method | Detailed Explanation |
|---|---|
| `csv.writer(file)` | Creates a CSV writer object that writes rows to the given file object. Automatically handles quoting fields that contain commas, newlines, or the quote character. Used to write the keyword CSV (step 2-1) and CVE details CSV (step 2-2 output). |
| `writer.writerow([values])` | Writes a single row to the CSV file. Accepts a list of values. Automatically converts each value to a string and adds a newline at the end of the row. |
| `writer.writeheader()` | Available on DictWriter — writes the header row using the fieldnames specified during writer construction. Ensures the CSV has labeled columns for easy reading. |
| `csv.DictWriter( file, fieldnames=[...])` | A CSV writer that accepts dicts instead of lists. Each dict key must correspond to a fieldname. Used to write CVE data where each CVE is already stored as a dict with id, description, severity, cvss_score, and keyword fields. |

## 2.12 os (Operating System Interface — Standard Library)

| Install | Built-in Python standard library — no installation required |
|---|---|

| Function | Detailed Explanation |
|---|---|

| | |
|---|---|
| `os.makedirs( path, exist_ok=True)` | Creates a directory and all intermediate-level directories. exist_ok=True prevents FileExistsError if the directory already exists — making it safe to call repeatedly. Used to create the design_docs directory if it doesn't exist before checking for PDFs. |
| `os.listdir(path)` | Returns a list of all file and directory names in the specified path. Used with a generator expression to check whether any .pdf files exist in the design_docs directory, determining whether to run the real pipeline or demo mode. |
| `os.getenv( 'VARNAME', 'default')` | Reads an environment variable by name. If the variable is not set, returns the default value. Used to read API keys (PINECONE_API_KEY, NVD_API_KEY) from the environment without hardcoding them into source code — a critical security practice. |

## 2.13 time (Standard Library)

| Install | Built-in Python standard library — no installation required |
|---|---|

| Function | Detailed Explanation |
|---|---|
| `time.sleep(seconds)` | Pauses program execution for the specified number of seconds. Used between NVD API calls to respect rate limits: 6.5 seconds without an API key (5 requests per 30 seconds), 0.6 seconds with an API key (50 requests per 30 seconds). Without this, rapid repeated queries would result in HTTP 403 Forbidden responses from the NVD. |

## 2.14 warnings (Standard Library)

| Function | Detailed Explanation |
|---|---|
| `warnings.filterwarnings ('ignore')` | Suppresses all Python warning messages for cleaner console output during the demo. Without this, HuggingFace Transformers and PyTorch emit numerous deprecation notices and configuration warnings that can obscure the actual program output. Used at the top of each script for cleaner demo presentation. |

# 3. Custom Functions — Detailed Explanation

Every custom function written in the project is documented below, organized by file. Each entry explains the function's purpose, parameters, return value, internal logic, and design decisions.

## 3.1 task1_base_llm.py — Functions

### load_documents(pdf_dir: str) → str

| Purpose | Load all PDFs from a directory and return combined text for use as LLM context |
|---|---|
| **Parameter / Return** | **Details** |
| `pdf_dir: str` | Path to directory containing design document PDF files |
| `Returns: str` | Single string of combined text from first 30 chunks (approx. 15,000 characters) |

Logic: Uses PyPDFDirectoryLoader to load all PDFs simultaneously, then RecursiveCharacterTextSplitter to split into 500-character chunks with 20-character overlap. The first 30 chunks are concatenated with double newlines as the context string. The 30-chunk limit prevents prompt overflow — Llama 2 7B has a 4,096-token context window. In the base LLM approach, this entire string is injected raw into the prompt without retrieval.

### load_model(model_id: str) → pipeline

| Purpose | Download, configure, and return a Llama 2 inference pipeline |
|---|---|
| **Parameter / Return** | **Details** |
| `model_id: str` | HuggingFace model identifier e.g. 'meta-llama/Llama-2-7b-chat-hf' |
| `Returns: pipeline` | HuggingFace text-generation pipeline object ready for inference |

Logic: Loads tokenizer with AutoTokenizer.from_pretrained(), then loads model with AutoModelForCausalLM.from_pretrained() using float16 on GPU or float32 on CPU. load_in_8bit=True enables bitsandbytes quantization when a GPU is present, reducing the 7B model from ~14GB (float16) to ~7GB (8-bit). Wraps both in a pipeline with temperature=0.1 for factual determinism and repetition_penalty=1.1 to prevent repetitive loops.

### build_prompt(context: str, question: str) → str

| Purpose | Construct a properly formatted Llama-2-Chat prompt string |
|---|---|
| **Parameter / Return** | **Details** |
| `context: str` | Concatenated document text chunks to provide as context |
| `question: str` | The threat modeling question to answer |

| | |
|---|---|
| `Returns: str` | Formatted prompt string following Llama-2-Chat template |

Logic: Assembles the four-part Llama-2-Chat template: start token <s>, [INST] marker, <<SYS>> system message with security expert persona, the context+question, and [/INST] closing tag. The system message instructs the model to answer from context only and be specific and concise — critical for reducing hallucinations.

### query_base_llm(pipe, context: str, question: str) → str

| Purpose | Execute a single query through the base LLM and extract the answer |
|---|---|
| **Parameter / Return** | **Details** |
| `pipe` | HuggingFace text-generation pipeline from load_model() |
| `context: str` | Document context string |
| `question: str` | Question to answer |
| `Returns: str` | The model's answer text only (not including the prompt) |

Logic: Builds the prompt with build_prompt(), passes it through the pipeline, and extracts only the generated answer by splitting on [/INST] and taking the last part. The pipeline returns the full text including the prompt; splitting on [/INST] cleanly separates prompt from generated content since Llama 2 appends its response immediately after [/INST].

### simulate_base_llm_answer(question: str, context: str) → str

| Purpose | Demo stub returning pre-written verbose answers without needing real Llama 2 |
|---|---|

Logic: Maps question keywords to pre-written answer strings that accurately represent the verbose, sometimes generic nature of base LLM responses. Includes a fallback for unrecognized questions. Allows the demo to run without GPU or HuggingFace credentials. Answers average ~120 words, reflecting the paper's finding that base LLM averages ~253 words.

## 3.2 task1_rag_llm.py — Functions

### load_and_chunk(pdf_dir: str) → list[Document]

| Purpose | Load PDFs and split into semantically-meaningful chunks with metadata |
|---|---|

Logic: Combines document loading and splitting in one function. Returns a list of LangChain Document objects — each has page_content (the chunk text, max 500 chars) and metadata dict containing source filename and page number. The metadata is preserved through all downstream operations, enabling answer provenance tracking.

### create_embeddings() → HuggingFaceEmbeddings

| Purpose | Load the sentence-transformer embedding model |
|---|---|

Logic: Instantiates HuggingFaceEmbeddings with all-MiniLM-L6-v2. On first call, downloads the model (~80MB) from HuggingFace Hub to local cache. Subsequent calls use the cache. Returns the embeddings object that both FAISS and Pinecone vector stores use to embed documents and queries.

### build_vector_store(chunks, embeddings) → retriever

| Purpose | Build a Pinecone vector store from document chunks, fall back to FAISS if unavailable |
|---|---|

Logic: First tries to initialize Pinecone with the API key from environment variables. Checks if the target index already exists before creating a new one (idempotent). Pushes all chunk embeddings to Pinecone and returns an as_retriever() object with k=5. If Pinecone initialization fails (no API key, network error), catches the exception and calls build_faiss_store() as fallback, ensuring the demo always works.

### build_faiss_store(chunks, embeddings) → retriever

| Purpose | Build a local in-memory FAISS vector store as fallback |
|---|---|

Logic: Creates a FAISS index entirely in local memory using FAISS.from_documents(). No API key or network connection required. Identical retriever interface to Pinecone, so all downstream code works unchanged. Data does not persist between runs — the index is rebuilt from documents each execution.

### build_rag_prompt(retrieved_docs: list, question: str) → str

| Purpose | Build Llama-2-Chat prompt using only retrieved document chunks as context |
|---|---|

Logic: Joins the page_content of all retrieved Documents with double newlines to form the context. Unlike the base LLM approach that uses all 30 chunks, RAG injects only the 5 most semantically relevant chunks — dramatically reducing prompt length and focusing the model on pertinent information. The system message explicitly instructs the model to say it does not know if context is insufficient.

### rag_query(pipe, retriever, question: str) → dict

| Purpose | Execute the complete RAG pipeline: retrieve → prompt → generate → extract |
|---|---|

| Return Key | Value |
|---|---|
| `answer` | The generated answer text extracted from the model output |
| `sources` | List of 'filename p.N' strings from retrieved document metadata |
| `word_count` | Integer count of words in the answer (avg ~74 for RAG) |
| `chunks_retrieved` | Number of chunks retrieved (equals k=5) |

Logic: Calls retriever.get_relevant_documents(question) to get top-5 chunks. Extracts source+page metadata from each. Builds the grounded prompt with build_rag_prompt(). Passes through LLM

pipeline. Splits output on [/INST] to extract answer. Returns a structured dict enabling comparison metrics (word count, sources, etc.).

### extract_keywords(text: str, top_n: int = 15) → list

| Purpose | Extract top-N semantic keywords using KeyBERT with dual stopword filtering |
|---|---|

Logic: Combines NLTK English stopwords (179 words) with the expert-defined stopword set (security-domain terms that are too common to be useful). Calls KeyBERT.extract_keywords() with MMR diversity=0.5. Returns a flat list of keyword strings (no scores), ready for NVD API queries.

## 3.3 task2_base_llm.py — Functions

### build_threat_prompt(context: str, question: str) → str

| Purpose | Build a security-focused Llama-2-Chat prompt for threat identification |
|---|---|

Logic: Similar to task1's build_prompt() but the system message persona changes from 'design understanding assistant' to 'cybersecurity expert conducting threat modeling'. This persona instruction shifts the model's output toward threat-focused vocabulary (attack vectors, CVEs, CVSS, mitigations) rather than system description vocabulary.

### query_base_llm(pipe, question: str) → str

| Purpose | Query base LLM for threat identification using embedded design context |
|---|---|

Logic: Uses the module-level DESIGN_CONTEXT constant (a multi-line string describing the system architecture) instead of loading external PDFs. This simulates the scenario where the context is small enough to fit directly in the prompt. Passes context + question to build_threat_prompt(), runs through the pipeline, extracts the answer after [/INST].

### simulate_base_llm_threat_answer(question: str) → str

| Purpose | Demo stub returning realistic verbose threat answers without model inference |
|---|---|

Logic: Six pre-written threat analysis answers, one per MQ2 question. Each answer is deliberately verbose (80-130 words) and generic — matching the paper's finding that base LLM responses lack specific CVE references. Covers Diffie-Hellman attacks, Modbus risks, open-source auth concerns, replay attack mitigations, MITM vectors, and private key exposure scenarios.

## 3.4 task2_rag_nvd.py — Functions (Main Contribution)

### extract_keywords(text: str, top_n: int) → list

| Purpose | Full NLP keyword extraction pipeline with stopword removal, KeyBERT, and stemming deduplication |
|---|---|

Logic: (1) Merges NLTK English stopwords with EXPERT_STOPWORDS set. (2) Over-fetches top_n*2 keywords to have candidates after deduplication. (3) Calls KeyBERT with MMR diversity=0.6 (slightly higher than task1 for more varied NVD search terms). (4) Applies PorterStemmer to the first word of each keyphrase. (5) Uses a seen_stems set to deduplicate — if 'encrypt' is already in seen_stems, neither 'encrypted' nor 'encryption' will appear. Returns a clean list of top_n diverse, meaningful keyword strings.

### save_keywords_csv(keywords: list, filepath: str)

| Purpose | Persist extracted keywords to a CSV file for inspection and reproducibility |
|---|---|

Logic: Creates a CSV with a 'keyword' header column. Each keyword is written as one row. Allows security engineers to review and manually add 'expert-identified' keywords (as described in the paper) before the NVD query stage. Provides an audit trail of what keywords drove CVE retrieval.

### query_nvd(keyword: str, results_per_page: int = 20) → list

| Purpose | Query NVD REST API v2 for CVEs matching a single keyword, return structured list |
|---|---|

| Return Field | Source in NVD Response |
|---|---|
| `id` | cve.id — the CVE-YYYY-NNNNN identifier |
| `description` | cve.descriptions[lang=='en'].value — English language description |
| `severity` | cve.metrics.cvssMetricV31[0].cvssData.baseSeverity — NONE/LOW/MEDIUM/HIGH/CRITICAL |
| `cvss_score` | cve.metrics.cvssMetricV31[0].cvssData.baseScore — float 0.0-10.0 |
| `keyword` | The search keyword that returned this CVE (for metadata tracking) |

Logic: Builds params dict with keywordSearch=keyword and resultsPerPage. Optionally adds the NVD API key to headers for higher rate limits. Sends GET request with timeout=10. On success, iterates vulnerabilities array, extracts English description, and tries CVSS v3.1, v3.0, v2 metrics in order (falls back gracefully if a version is missing). Returns empty list on network failure without crashing the program.

### build_vulnerability_dataset(keywords: list, output_csv: str) → list

| Purpose | Query NVD for all extracted keywords, deduplicate, and save complete CVE dataset |
|---|---|

Logic: Iterates over the keywords list, calling query_nvd() for each. Uses a seen_ids set to deduplicate CVEs that appear for multiple keywords (e.g., an OpenSSL CVE might appear for both 'openssl' and 'tls' searches). Sleeps between queries to respect NVD rate limits. Saves the deduplicated list to CSV with DictWriter. Returns the list for immediate use in the JSONL conversion step.

### cves_to_jsonl(cves: list, output_file: str) → str

| Purpose | Convert CVE list to JSONL format optimized for Pinecone vector DB ingestion |
|---|---|

Logic: For each CVE dict, creates a JSONL entry with two keys: 'text' (the searchable string, formatted as 'CVE-ID: description') and 'metadata' (cve_id, severity, cvss_score, keyword). The text field is what gets embedded — combining the CVE ID with its description ensures that both ID-based searches and semantic description searches find the entry. Writes one json.dumps() + newline per CVE.

### load_jsonl(filepath: str) → list

| Purpose | Read JSONL file back into Python list of dicts, handling empty lines gracefully |
|---|---|

Logic: Reads file line by line. Strips whitespace and skips empty lines. Calls json.loads() on each non-empty line. If a line is malformed JSON, the exception propagates (caller handles). The line-by-line reading is memory-efficient — the entire file is never loaded into RAM at once.

### build_vuln_vector_store(jsonl_path: str, embeddings) → retriever

| Purpose | Embed all CVE descriptions and build a searchable vector store |
|---|---|

Logic: Loads JSONL entries, converts each to a LangChain Document with page_content=text (the CVE ID + description) and metadata=metadata (severity, CVSS score, etc.). Tries Pinecone first, falls back to FAISS. The metadata stored with each vector means that when a CVE is retrieved, its severity and CVSS score are immediately available alongside the description — no secondary lookup needed.

### build_threat_rag_prompt(retrieved_cves: list, question: str) → str

| Purpose | Build threat-focused RAG prompt that includes CVE severity context alongside descriptions |
|---|---|

Logic: Formats each retrieved CVE as a text block: the CVE description followed by 'Severity: HIGH | CVSS: 7.5'. This severity context is crucial — it tells the LLM which vulnerabilities are most critical and should be prioritized in the answer. The system message instructs the model to reference CVE IDs in its response, enabling traceable, auditable threat assessments.

### rag_threat_query(pipe, retriever, question: str) → dict

| Purpose | Complete RAG threat identification pipeline: retrieve CVEs → prompt → generate → extract |
|---|---|

| Return Key | Value |
|---|---|
| `answer` | Concise CVE-grounded threat answer (avg ~74 words) |
| `cve_ids` | List of CVE-YYYY-NNNNN strings from retrieved CVE metadata |
| `severities` | List of 'CVE-ID (SEVERITY / CVSS X.X)' strings for display |
| `word_count` | Integer word count of the answer |

Logic: Retrieves top-5 CVEs via similarity search on the question. Extracts CVE IDs and severity strings from metadata. Builds the threat RAG prompt. Generates with Llama 2. Extracts answer. Returns all metadata needed for evaluation (CVE references, severity context, word count for comparison with base LLM).

### get_mock_cves() → list

| Purpose | Return 15 real CVEs hard-coded for demo mode, covering all system components |
|---|---|

Logic: Returns a Python list of 15 actual, real CVEs selected to match the smart grid demo system: OpenSSL vulnerabilities (CVE-2023-0286, CVE-2022-0778, CVE-2021-3449), Diffie-Hellman issues (CVE-2022-40735, CVE-2002-20001), AES problems (CVE-2023-1255), authentication flaws (CVE-2022-21449, CVE-2021-44228), Modbus risks (CVE-2021-30965, CVE-2019-13945), Intel TDX side channels (CVE-2022-21123, CVE-2022-0001, CVE-2023-23583). All CVE IDs and descriptions are accurate as of the knowledge cutoff.

## 3.5 demo_runner.py — Functions

### log(msg: str = '')

| Purpose | Print to console AND append to output_lines list for file saving |
|---|---|

Logic: Calls print() for immediate console visibility and output_lines.append() to accumulate all output. At the end of main(), the full output_lines list is written to demo_output.txt. This dual-output pattern ensures the demo is both interactive (real-time console) and archiveable (saved text file for submission).

### run_task1_base() / run_task1_rag() / run_task2_base() / run_task2_rag()

| Purpose | Execute each of the four approaches and log formatted output with evaluation labels |
|---|---|

Logic: Each function imports simulation functions from the corresponding module, runs all 6 questions, and logs question, answer, and metadata (word count, sources, CVE IDs). run_task2_rag() also demonstrates the intermediate pipeline steps (keyword extraction, NVD query, JSONL creation, vector store building) so viewers can see the full pipeline in action.

### print_comparison()

| Purpose | Print a formatted comparison table of Base LLM vs RAG-Enhanced LLM across 8 metrics |
|---|---|

Logic: Uses Python string formatting (f-strings with width specifiers) to align three columns: metric name, base LLM value, RAG LLM value. Metrics are drawn directly from the paper's evaluation section: response length (~253 vs ~74 words), satisfaction rate (65% vs >75%), CVE references (never vs always), etc. Provides a concise summary that mirrors Table IV in the paper.

# 4. Configuration Constants — Explanation

Every module-level constant is intentional. The table below explains why each value was chosen.

| Constant | Value | Reason for This Value |
|---|---|---|
| `CHUNK_SIZE` | 500 | Balances context richness (more text = more context) vs. embedding quality (shorter text = more precise embedding). 500 chars ≈ 3-4 sentences — enough for a complete design concept. |
| `CHUNK_OVERLAP` | 20 | Small overlap (≈1 sentence fragment) prevents context from being cut at exactly the wrong point. 20 chars avoids losing a key term at a chunk boundary. |
| `TOP_K` | 5 | Retrieves 5 chunks per query. Enough to give diverse context without overwhelming the LLM prompt or introducing noise from less-relevant chunks. |
| `TOP_N_KEYWORDS` | 15 | Extracts 15 keywords per document for NVD queries. Enough to cover major system components (TLS, AES, DH, Modbus, Intel TDX) without producing too many CVEs to manage. |
| `MAX_NEW_TOKENS (Base LLM)` | 512-600 | Base LLM is allowed more tokens since it has no retrieval — more context means longer reasoning is needed. Matches observed ~253-word average. |
| `MAX_NEW_TOKENS (RAG LLM)` | 300 | RAG LLM is restricted to encourage conciseness. With relevant context already retrieved, the model needs fewer tokens to answer accurately. |
| `TEMPERATURE` | 0.1 | Near-zero temperature makes output nearly deterministic. Critical for security analysis where reproducibility and factual accuracy outweigh creativity. |
| `REPETITION PENALTY` | 1.1-1.15 | Penalizes tokens that have appeared recently. Prevents the model from repeating the same phrase or restating context verbatim. Higher for RAG (1.15) since retrieved text is already present. |
| `DIVERSITY (KeyBERT)` | 0.5-0.6 | MMR diversity parameter. Task 1 uses 0.5 (balanced) for document understanding. Task 2 uses 0.6 (more diverse) to produce a wider variety of NVD search keywords across different vulnerability types. |

# 5. Complete Data Flow Diagram (Text)

## 5.1 Task 1 — Base LLM Data Flow

```
design_docs/*.pdf
      |
      ▼  PyPDFDirectoryLoader.load()
List[Document]  ← {page_content, metadata:{source, page}}
      |
      ▼  RecursiveCharacterTextSplitter.split_documents()
List[Document] chunks  ← 500-char chunks, 20-char overlap
      |
      ▼  concat first 30 chunks
context_string  ← raw text, no retrieval
      |
      ▼  build_prompt(context_string, question)
prompt_string  ← Llama-2-Chat template
      |
      ▼  pipeline(prompt_string)
raw_output  ← full text including prompt
      |
      ▼  .split('[/INST]')[-1].strip()
answer_string  ← ~253 words, generic, no sources
```

## 5.2 Task 1 — RAG LLM Data Flow

```
INDEXING PHASE (done once per document set):
design_docs/*.pdf
      |  PyPDFDirectoryLoader + RecursiveCharacterTextSplitter
      ▼
List[Document] chunks
      |  HuggingFaceEmbeddings (all-MiniLM-L6-v2)
      ▼
List[float[384]] embeddings
      |  FAISS.from_documents() OR Pinecone.from_documents()
      ▼
Vector Store (indexed, searchable)

QUERY PHASE (per question):
question_string
      |  embeddings.embed_query()
      ▼
query_vector[384]
      |  retriever.get_relevant_documents() → cosine similarity
      ▼
List[Document] top-5 chunks  ← most relevant to question
      |  build_rag_prompt(retrieved_docs, question)
      ▼
grounded_prompt_string  ← only relevant context
      |  pipeline(grounded_prompt_string)
      ▼
answer_string  ← ~74 words, concise, source-traced
```

## 5.3 Task 2 — RAG + NVD Data Flow

```
design_docs/*.pdf  (or DESIGN_CONTEXT string)
     │
     ▼  STEP 2-1: KeyBERT.extract_keywords()
keywords = ['diffie-hellman', 'aes-256', 'modbus tcp', ...]
     │  save_keywords_csv()
     ▼
keywords.csv
     │
     ▼  STEP 2-2: query_nvd() for each keyword → NVD REST API v2
List[CVE dicts] = [{id, description, severity, cvss_score}, ...]
     │  Deduplicate by CVE ID  |  time.sleep() between queries
     ▼
cve_details.csv  ← all unique CVEs
     │
     ▼  STEP 2-3: cves_to_jsonl()
vulnerabilities.jsonl  ← one JSON per line: {text, metadata}
     │
     ▼  STEP 2-4: load_jsonl() → Document objects
     │  HuggingFaceEmbeddings → embed each CVE description
     ▼
Pinecone / FAISS Vector Store  ← 1,000-36,000 CVE vectors
     │
QUERY PHASE:
threat_question
     │  embed_query() → cosine similarity search
     ▼
top-5 CVE Documents  ← {description, severity, cvss_score}
     │  build_threat_rag_prompt(retrieved_cves, question)
     ▼
threat_prompt  ← CVE IDs + descriptions + severity in context
     │  Llama 2 pipeline()
     ▼
threat_answer  ← ~40-74 words, CVE-grounded, severity-aware
```

# 6. Sample Outputs With Annotations

## 6.1 Task 1: RAG vs Base LLM — 'How is data secured during transmission?'

### Base LLM Output (verbose, generic)

```
Data security during transmission is a multi-layered concern.
Encryption algorithms like AES-256, ChaCha20, and 3DES are commonly
used in industry. TLS provides both encryption and authentication.
Certificate pinning prevents MITM attacks. Message integrity is
ensured through HMAC or digital signatures. Regular key rotation is
a best practice to limit exposure. The system uses AES-256 based on
the documentation provided.

[Word count: 72 | No source attribution | Generic industry advice]
```

### RAG-Enhanced LLM Output (concise, source-traced)

```
Data is secured with AES-256 encryption during transmission.
Message integrity is enforced via RSA-2048 signatures from the
Aggregator. TLS 1.3 provides transport-layer security across all
connections.

Sources: ['design_doc.pdf p.5', 'design_doc.pdf p.9']
[Word count: 31 | Source-attributed | Specific to this system]
```

## 6.2 Task 2: RAG+NVD vs Base LLM — 'Modbus TCP/IP risks?'

### Base LLM Output (no CVE IDs)

```
Modbus TCP/IP is inherently insecure: (1) No authentication.
(2) No encryption. (3) No message integrity. (4) Replay attacks
trivial without sequence numbers. (5) DoS by request flooding.
(6) Function code abuse enables physical manipulation.
Mitigation: segmentation, OT firewalls, encrypted tunnels.

[Word count: 47 | No CVE references | Generic advice]
```

### RAG+NVD Output (CVE-grounded with CVSS scores)

```
CVE-2021-30965 (HIGH, CVSS 8.8): Modbus stack buffer overflow —
network attackers crash devices or achieve RCE.
CVE-2019-13945 (HIGH, CVSS 7.5): SIMATIC improper Modbus input
validation — malformed packets cause DoS. Modbus has zero built-in
auth or encryption. Network segmentation and OT firewalls essential.

CVEs: ['CVE-2021-30965', 'CVE-2019-13945']
Severity: ['CVE-2021-30965 (HIGH/8.8)', 'CVE-2019-13945 (HIGH/7.5)']
```

```
[Word count: 42 | CVE-grounded | Actionable severity context]
```

## 6.3 RAG 'Awareness of Honesty' — When Context is Insufficient

```
Q: How is the custom Modbus TCP/IP protocol stack implemented?

A (RAG): Based on the provided context, I don't know the answer
to your question. The passage does not provide any information
about how the custom Modbus TCP/IP protocol stack is implemented.
The authors seem focused on use of Modbus TCP in industrial
processes rather than detailing the protocol stack implementation.

[This behaviour is unique to RAG — base LLM would hallucinate an answer]
```

# 7. Evaluation Metrics — Detailed Breakdown

| Metric | Base LLM | RAG-Enhanced LLM |
|---|---|---|
| Avg response words | ~253 | ~74 |
| Human eval pass rate | ~65% | >75% |
| CVE ID references | Never | Always |
| Hallucinations | Higher | Reduced |
| Honesty when uncertain | No | Yes |
| Uses live NVD data | No | Yes |
| Source attribution | None | Doc + Page |
| Answer conciseness | Verbose | Concise |
| Query response time | 25–50s | 25–50s |
| Token prompt size | Large (30 chunks) | Small (5 chunks) |

The human evaluation process used two independent coders who rated each answer as '+' (meets expectations) or '-' (does not). Both coders coded independently then consolidated results. The evaluation scope covered 12 design documents × 6 questions = 72 total query evaluations. Questions were classified into: system description, system environment, data handling (Task 1); secrets management, identity & access control, network configurations (Task 2).

# 8. Quick Reference — All Library Methods Used

| Library | Method / Class | One-Line Description |
|---|---|---|
| transformers | AutoTokenizer.from_pretrained() | Load tokenizer for specified model from HuggingFace Hub |
| transformers | AutoModelForCausalLM.from_pretrained() | Load causal LM weights with dtype + quantization options |
| transformers | pipeline('text-generation', ...) | High-level inference wrapper with generation parameters |
| torch | torch.cuda.is_available() | Check if CUDA GPU is available for acceleration |
| torch | torch.float16 / float32 | Data type constants for model weight precision control |
| langchain | PyPDFDirectoryLoader(dir) | Load all PDFs from a folder into Document objects |
| langchain | loader.load() | Execute the PDF loading and return Document list |
| langchain | RecursiveCharacterTextSplitter(...) | Split documents at natural boundaries into chunks |
| langchain | splitter.split_documents(docs) | Apply splitting to Document list, preserve metadata |
| langchain | HuggingFaceEmbeddings(model_name) | Load sentence-transformer embedding model |
| langchain | FAISS.from_documents(docs, emb) | Build in-memory FAISS vector index from documents |
| langchain | Pinecone.from_documents(docs, emb) | Upload document vectors to Pinecone cloud index |
| langchain | vector_store.as_retriever(k=5) | Create retriever that returns top-k similar documents |
| langchain | retriever.get_relevant_documents(q) | Retrieve top-k chunks most semantically similar to query |
| keybert | KeyBERT(model=...) | Initialize keyword extractor with embedding model |
| keybert | kw_model.extract_keywords(text, ...) | Extract top-N semantically relevant keywords from text |
| nltk | nltk.download('stopwords') | Download English stopword list corpus |
| nltk | stopwords.words('english') | Get 179 common English stopwords to filter out |
| nltk | PorterStemmer() | Initialize rule-based stemmer for root word extraction |

| Library | Method / Class | One-Line Description |
|---|---|---|
| nltk | stemmer.stem(word) | Reduce a word to its root form for deduplication |
| pinecone | pinecone.init(api_key, env) | Authenticate Pinecone client with credentials |
| pinecone | pinecone.list_indexes() | List all existing indexes in the account |
| pinecone | pinecone.create_index(name, dim, metric) | Create new cosine-similarity vector index |
| requests | requests.get(url, params, headers, timeout) | Send HTTP GET to NVD API with query parameters |
| requests | response.raise_for_status() | Raise exception on HTTP 4xx/5xx error responses |
| requests | response.json() | Parse JSON response body into Python dict |
| json | json.dumps(obj) | Serialize Python dict to JSON string for JSONL writing |
| json | json.loads(string) | Parse JSONL line string back to Python dict |
| csv | csv.writer(file) | Create CSV writer for keyword/CVE output files |
| csv | writer.writerow([values]) | Write one row to CSV file |
| csv | csv.DictWriter(file, fieldnames) | Write dicts as CSV rows using field names |
| os | os.makedirs(path, exist_ok=True) | Create directory tree, no error if exists |
| os | os.listdir(path) | List all files in directory for PDF detection |
| os | os.getenv('KEY', 'default') | Read environment variable with fallback default |
| time | time.sleep(seconds) | Pause execution for NVD API rate limit compliance |
| warnings | warnings.filterwarnings('ignore') | Suppress HuggingFace/PyTorch deprecation warnings |

— End of Document —

Group 10 | LLM-Based Threat Modeling | AISCC 2024